BOOTCAMP AI WEB PROGRAMMING

Aplikasi Manajemen Bengkel

Membangun aplikasi bengkel bersama Tim

Pertemuan 9



Peta Belajar

LANGKAH 1

Collaboration

Bekerja dalam kelompok

LANGKAH 2

Branch dan *pull request*

Menggabungkan kerja tim

LANGKAH 3

Aplikasi bengkel

Gambaran yang dibangun

LANGKAH 4

Fitur per anggota

Empat fitur dikerjakan bersamaan

LANGKAH 5

Integrasi

Menguji alur lengkap

LANGKAH 6

Uji silang dan tayang

Diuji kelompok lain

Hasil hari ini: aplikasi Bengkel Cemerlang Jaya Motor buatan kelompok, tayang di Netlify.

SOFT SKILL

Collaboration

Bekerja dalam satu kelompok



Bagaimana Aplikasi Dibangun di Industri?

SOFT SKILL
Collaboration

Di Industri, aplikasi dibangun oleh tim. Hari ini empat orang membangun empat fitur **bersamaan** di satu aplikasi dan satu database. Tanpa aturan, tiga hal ini sering terjadi:

Kode saling menimpa	Dua orang mengubah file yang sama, hasil kerja salah satunya hilang.
Database berubah tanpa kabar	Tabel diubah satu orang, fitur anggota lain tiba-tiba error.
Fitur tidak tersambung	Setiap fitur jalan sendiri, tetapi gagal saat digabung.

Apa Itu Kolaborasi?



SOFT SKILL
Collaboration

DEFINISI

Kolaborasi adalah bekerja bersama orang lain untuk tujuan yang sama, dengan berbagi tugas, informasi, dan tanggung jawab atas hasilnya.

Contohnya tiga bentuk kolaborasi yang dipakai hari ini:

Pull request	Meminta hasil kerjamu digabungkan ke kode utama.
Code review	Teman membaca perubahanmu sebelum digabungkan.
Pair testing	Dua orang menguji satu aplikasi bersama dan mencatat temuannya.

Siapa Mengerjakan Apa di Kelompok?

SOFT SKILL
Collaboration

Satu kelompok berisi empat orang. Setiap anggota memegang **satu fitur**. Salah satunya juga menjadi ketua.

A	B	C	D
Work Order	Tagihan dan *Invoice*	Inventory dan Pengguna	*Dashboard* dan *Audit Log*

Tugas tambahan ketua

- Menyiapkan repo GitHub dan proyek Supabase kelompok
- Menggabungkan *pull request* ke main
- Menayangkan aplikasi ke Netlify di akhir sesi

Apa Itu *Branch*?

DEFINISI

Branch adalah jalur kerja terpisah di dalam satu repo. Perubahan di sebuah *branch* tidak mengganggu kode utama (main) sampai digabungkan.

Contohnya empat *branch* di repo kelompok:



Apa Itu *Pull Request*?

DEFINISI

Pull request (PR) adalah permintaan untuk menggabungkan perubahan dari sebuah *branch* ke main. Di PR, teman membaca perubahan, memberi komentar, lalu menyetujui.



1. **Judul dan deskripsi**
   Apa yang dibuat dan cara mengujinya.

2. **File yang berubah**
   Teman membaca perubahan di tab ini.

3. **Persetujuan**
   Setelah disetujui, ketua menekan Merge.

Bagaimana Alur Kerja Git Kelompok?

1. Buat *branch* fitur dari main, misalnya fitur/tagihan.
2. Kerjakan fitur, lalu *commit* setiap bagian selesai.
3. *Push branch* ke GitHub.
4. Buat *pull request* dan pilih satu teman sebagai *reviewer*.
5. *Reviewer* membaca perubahan, mencoba, lalu menyetujui.
6. Ketua menggabungkan PR ke main.
7. **Setelah ada PR yang digabungkan, semua anggota menarik main terbaru ke laptopnya.**

Langkah 1 sampai 3 dan 7 cukup diminta ke agen lewat prompt. Langkah 4 sampai 6 di GitHub.

Apa Itu *Merge Conflict*?

DEFINISI
Merge conflict adalah keadaan ketika dua *branch* mengubah baris yang sama di satu file, sehingga Git tidak tahu versi mana yang dipakai.

Contohnya anggota A dan C sama-sama mengubah judul menu:

```
<<<<<<< fitur/work-order
  label: 'Work Order Aktif',
=======
  label: 'Daftar WO',
>>>>>>> fitur/inventory
```

Agen bisa membantu memilih atau menggabungkan kedua versi.

Cara mencegahnya:

1. Satu fitur bekerja di folder halamannya sendiri.
2. Kabari kelompok sebelum mengubah file bersama, misalnya sidebar.
3. Tarik main terbaru sebelum membuat *pull request*.

Bagaimana Memakai Satu Database Bersama?

Kode punya *branch*, tetapi database tidak. Keempat anggota memakai **satu proyek Supabase** yang sama, jadi perubahan seseorang langsung terasa oleh semua.

1 Kabari dulu

Sampaikan ke kelompok sebelum menyetujui SQL dari agen: "Aku mau menambah fungsi tambah_item_tagihan."

2 Simpan SQL-nya

Minta agen menyimpan SQL ke supabase/migrations/ supaya ikut terlihat di *pull request*.

3 Jangan ubah tabel

Tabel sudah lengkap di starter. Setiap fitur hanya menambah *database function*.

4 Perbarui tipe

Setelah ada perubahan, buat ulang src/types/database.ts lewat agen.

Bagaimana Memberi Kabar ke Kelompok?

SOFT SKILL
Collaboration

Kelompok duduk berdekatan, jadi kabar disampaikan **langsung**. Sampaikan di tiga saat ini:

Sebelum ubah database

"Aku mau menyetujui fungsi tagihan, ya."

Saat terhambat

"Tombol status error. Ada yang bisa bantu lihat?"

Saat PR siap

"PR fitur C siap. Siapa yang bisa review?"

Cek singkat 3 menit,

Ketua memimpin. Setiap anggota menyebut tiga hal:

Sudah **Sedang** **Terhambat**

Bagaimana Mereview *Pull Request* Teman?

SOFT SKILL
Collaboration

Reviewer tidak harus membaca setiap baris kode. Periksa empat hal ini:

- [ ] Deskripsi PR menjelaskan apa yang dibuat dan cara mengujinya.
- [ ] Fiturnya dicoba di laptopmu dan berjalan.
- [ ] Tidak ada file milik fitur lain yang ikut berubah.
- [ ] Tidak ada isi .env.local atau *secret key* di perubahan.

Contoh komentar yang membantu:
"Tombol Terbitkan Invoice masih muncul untuk Pak Joko. Bisa dicek lagi?"

BAGIAN 2

Mengenal Aplikasi Bengkel



Gambaran aplikasi yang dibangun hari ini

Apa Masalah Bengkel Cemerlang Jaya Motor?

Cemerlang Jaya Motor adalah bengkel mobil dengan satu Admin, dua Kepala Mekanik yang masing-masing memimpin satu tim, dan satu Owner. Selama ini pekerjaan dicatat di kertas.

Catatan Servis
B 1234 KJ · Avanza
Aki soak, ganti aki
Mekanik: Andi
Bayar: ?

B 2210 TFA · Xenia
Rem bunyi
Tunggu kampas rem

D **Pak Darto · Owner**

Mobil mana yang terlambat selesai?

Berapa pendapatan hari ini?

Spare part apa yang hampir habis?

DEFINISI

Work order (WO) adalah satu dokumen untuk satu kunjungan mobil, dari mobil datang, dikerjakan, diperiksa, ditagih, sampai diambil pemiliknya.

Aplikasi hari ini mencatat setiap kunjungan mobil dalam satu WO.

Siapa Saja Pengguna Aplikasi Bengkel?

Aplikasi dipakai tiga peran. Setiap peran punya pekerjaan dan batas yang berbeda:

DA **Pak Darto**
Owner

`owner@cemerlangjaya.test`

- Melihat semua data, harga modal, dan laba
- Mengelola pengguna, harga, dan stok
- Membaca *audit log*

RI **Rina**
Admin

`admin@cemerlangjaya.test`

- Menerima mobil dan membuat WO
- Menerbitkan *invoice* dan mencatat pembayaran
- Menyerahkan mobil ke pemiliknya

JS **Pak Joko · Pak Slamet**
Kepala Mekanik

`joko@ · slamet@cemerlangjaya.test`

- Menunjuk mekanik di timnya
- Mengubah status dan mencatat spare part
- Hanya untuk WO milik timnya

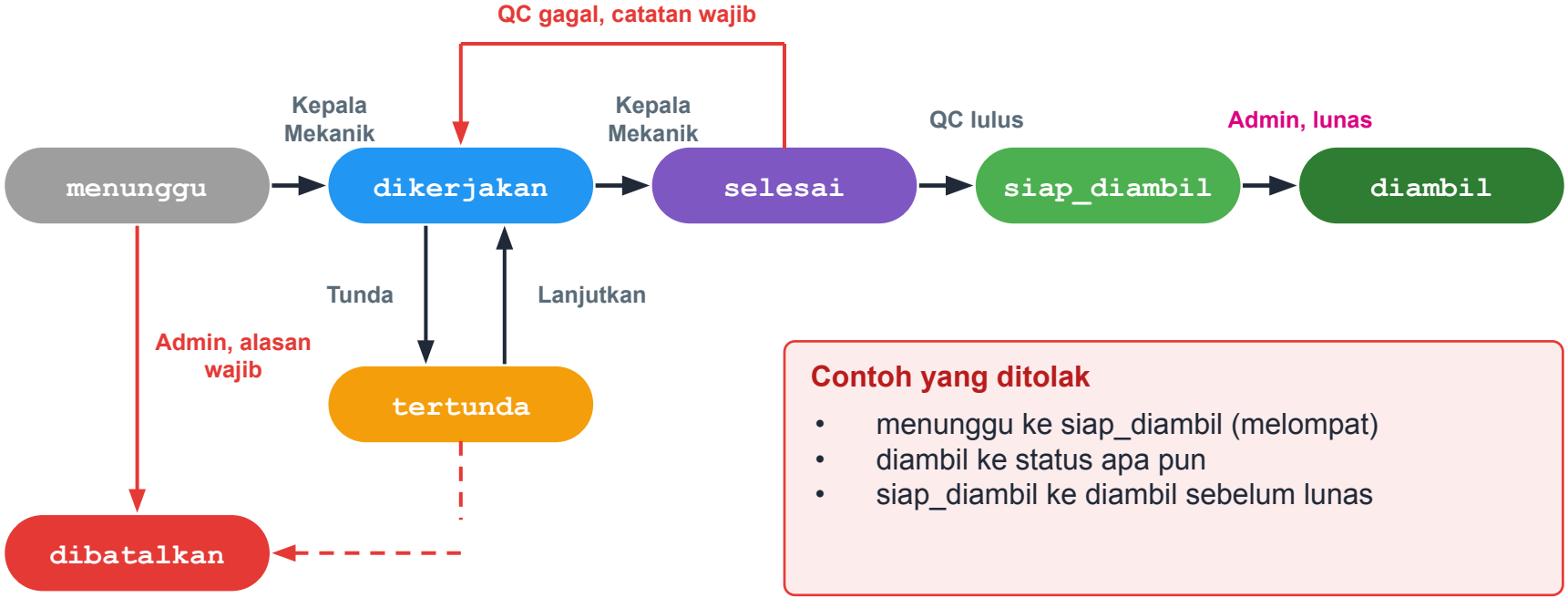
Mekanik tidak punya akun. Nama mekanik disimpan sebagai data tim: Andi dan Bayu di Tim A (Pak Joko), Dedi dan Eko di Tim B (Pak Slamet).

Bagaimana Mobil Diproses dari Datang sampai Diambil?



Bagaimana Status *Work Order* Berpindah?

Panah adalah perpindahan yang diizinkan beserta pelakunya. Perpindahan tanpa panah ditolak fungsi ubah_status_wo.



Contoh yang ditolak

- menunggu ke siap_diambil (melompat)
- diambil ke status apa pun
- siap_diambil ke diambil sebelum lunas

Siapa Boleh Melihat Harga Modal?

Harga jual dipakai semua peran untuk menagih. Harga modal dan laba hanya untuk Owner, jadi disimpan di **tabel terpisah**:

1 **inventory_items**

kode	nama	harga_jual	stok
SP-0004	Aki 45Ah	950.000	2
SP-0010	Lampu H4	85.000	1

Dibaca Owner, Admin, Kepala Mekanik

2 **inventory_costs**

item	harga_modal
SP-0004	780.000
SP-0010	55.000

Dibaca Owner saja

Rina membuka Inventory

- ✓ Harga jual tampil
- ✗ Kolom modal tidak ada
- ✗ Network: costs = []

1 **Tabel umum**
Kode, nama, harga jual, dan stok. RLS mengizinkan semua peran membaca.

2 **Tabel rahasia**
Hanya harga modal. RLS hanya mengizinkan Owner, jadi datanya tidak terkirim ke *browser* Rina.

Menyembunyikan kolom di tampilan tidak cukup. Datanya tidak boleh terkirim.

Apa Isi Starter App 4?

Pondasi yang dipakai bersama sudah jadi. Halaman empat fitur sudah ada, tetapi **baru kerangka**.

Sudah jadi

- ✓ Login, *route guard*, dan sidebar per peran
- ✓ Semua tabel, RLS, dan data contoh
- ✓ Halaman Pelanggan dan Tim Mekanik sebagai contoh
- ✓ dashboard.sql, PRD.md, dan AGENTS.md

Masih kerangka



Contohnya halaman Inventory di starter.

Tugasmu: melengkapi kerangka fiturmu, lalu menggabungkannya lewat *pull request*.

Bagaimana Fitur Dibagi di Kelompok?

Fitur	Halaman yang dilengkapi	*Database function* yang dibuat	*Branch*

A ***Work Order***	`/work-orders`
	`/work-orders/baru`
	`/work-orders/[id]`
		`buat_work_order`
		`tugaskan_mekanik`
		`ubah_status_wo`
			`fitur/work-order`

B **Tagihan dan *Invoice***	`/work-orders/[id]/tagihan`
	`/work-orders/[id]/invoice`
		`tambah_item_tagihan`
		`terbitkan_invoice`
		`catat_pembayaran, dll.`
			`fitur/tagihan`

C **Inventory dan Pengguna**	`/inventory`
	`/pengguna`
	`/setelan`
		`catat_stok_masuk`
		`sesuaikan_stok`
			`fitur/inventory`

D ***Dashboard* dan *Audit Log***	`/dashboard`
	`/audit-log`
	`/laporan-laba`
		`dashboard.sql`
		`trigger` `catat_audit`
			`fitur/dashboard`

Ketua juga memegang satu fitur. Tentukan pembagiannya sekarang, sebelum persiapan dimulai.

BAGIAN 3

Membangun Aplikasi Bengkel

Persiapan, fitur, dan integrasi



Langkah Persiapan Ketua Kelompok

1. Buka repo starter di GitHub, klik **Use this template › Create a new repository**. Nama: cemerlang-jaya-k1 (sesuai nomor kelompok).
2. Di repo baru, buka **Settings › Collaborators**, undang tiga anggota.
3. Di Supabase, buat proyek **cemerlang-jaya-k1**. Buka **Organization › Team › Invite**, undang tiga anggota sebagai **Administrator**.
4. *Clone* repo kelompok, buka di Antigravity, lalu kirim **Prompt K1**.
5. Buat empat akun uji di **Authentication › Users** dengan Auto Confirm User.
6. Kirim **Prompt K2**, lalu kabari kelompok: "Database siap."

LINK REPO STRATER

Prompt Persiapan Ketua

PROMPT K1 *salin teks di bawah ini*

Baca AGENTS.md dan docs/PRD.md dulu.
Pakai Supabase MCP untuk mengambil URL dan publishable key proyek cemerlang-jaya-k1. Salin .env.example menjadi .env.local lalu isi kedua nilainya tanpa menampilkannya di chat.
Setelah itu jalankan isi supabase/migrations/01_dasar.sql lewat apply_migration tanpa mengubahnya, lalu jalankan npm install dan npm run dev.

PROMPT K2 *salin teks di bawah ini*

Empat akun uji sudah aku buat. Lewat execute_sql, isi nama, peran, dan aktif = true keempat akun sesuai PRD 15.1. Lalu jalankan supabase/seed.sql tanpa mengubahnya, buat src/types/database.ts lewat generate_typescript_types, dan laporkan jumlah baris work_orders.

Akun uji: owner@, admin@, joko@, slamet@cemerlangjaya.test · kata sandi Bengkel123!

Langkah Persiapan Anggota

1. Terima undangan GitHub dan undangan Supabase di email.
2. *Clone* repo kelompok ke Local Disk D, lalu buka di Antigravity.
3. Pastikan Supabase di daftar MCP Servers aktif. Token dari Sesi 8 tetap dipakai.
4. Tunggu kabar "Database siap" dari ketua, lalu kirim **Prompt 0**.
5. Kirim **Prompt Branch** dengan nama *branch* fiturmu.
6. Masuk sebagai Pak Darto di localhost:3000 untuk memastikan semua berjalan.

Ketua juga mengirim Prompt Branch setelah K2 selesai.

Prompt Persiapan Anggota

PROMPT 0 *salin teks di bawah ini*

Baca AGENTS.md dan docs/PRD.md dulu.
Pakai Supabase MCP untuk mengambil URL dan publishable key proyek cemerlang-jaya-k1. Salin .env.example menjadi .env.local lalu isi kedua nilainya tanpa menampilkannya di chat. Buat src/types/database.ts lewat generate_typescript_types, lalu jalankan npm install dan npm run dev.

PROMPT BRANCH *salin teks di bawah ini*

Buat branch baru bernama fitur/tagihan dari main terbaru, lalu pindah ke branch itu. Tunjukkan nama branch yang aktif sekarang.

Ganti fitur/tagihan dengan *branch* fiturmu: fitur/work-order, fitur/tagihan, fitur/inventory, atau fitur/dashboard.

Bagaimana Bekerja dengan Agen dan Supabase MCP?

Pola yang sama dipakai setiap anggota di fiturnya:

1. **Kirim *prompt***
   Salin dari slide praktik
2. **Agen menampilkan SQL**
   Membaca PRD, belum menjalankan
3. **Kamu membaca SQL**
   Pakai daftar periksa di slide berikut
4. **Kabari lalu setujui**
   Agen menjalankan *apply_migration*
5. **Simpan SQL dan uji**
   Ke supabase/migrations, lalu coba dengan akun uji
6. ***Commit***
   Di *branch* fiturmu sendiri

Apa yang Diperiksa Sebelum Menyetujui SQL?

Contohnya *database function* yang ditampilkan agen untuk fitur C. Cari empat hal ini sebelum menyetujui:

```
create or replace function public.catat_stok_masuk(
  p_item uuid, p_jumlah int, p_keterangan text)
returns void language plpgsql
security definer set search_path = ''
as $$
declare i public.inventory_items;
begin
  if (select private.peran_saya()) not in
     ('owner', 'admin') then
    raise exception 'Hanya Admin atau Owner';
  end if;
  select * into i from public.inventory_items
    where id = p_item for update;
  ...
```

1. **Fungsi aman**
   security definer dan search_path kosong.
2. **Peran diperiksa di awal**
   Lewat private.peran_saya(), sebelum mengubah apa pun.
3. **Baris dikunci**
   for update pada data yang bisa direbut, misalnya stok.
4. **Nama sama dengan PRD**
   Nama fungsi dan parameternya persis PRD 9.2.

Tolak SQL yang mengubah tabel.

Fitur A: *Work Order*

A

Buat WO dari plat nomor, tunjuk mekanik, dan pindahkan status sesuai aturan PRD 7.2.

Fitur A

Halaman yang dilengkapi	*Database function* yang dibuat

`/work-orders`
`/work-orders/baru`
`/work-orders/[id]`
	`buat_work_order`
	`tugaskan_mekanik`
	`ubah_status_wo`
	trigger `jaga_wo`, *bucket* `foto-wo`

Yang dilatih di fitur ini

Status hanya berpindah lewat *database function*. Kepala Mekanik hanya bisa mengubah WO milik timnya, walaupun tombolnya dipanggil lewat Console.

Prompt Fitur A

PROMPT A1 *salin teks di bawah ini*

Fitur A. Lewat Supabase MCP, buat fungsi buat_work_order, tugaskan_mekanik, dan ubah_status_wo sesuai PRD 9.2 dan aturan perpindahan PRD 7.2, trigger jaga_wo (PRD 9.4), serta bucket foto-wo beserta policy-nya (PRD Bab 11). Setiap fungsi memeriksa peran dan tim di awal dan mengunci baris dengan for update. Tampilkan SQL dulu dan tunggu persetujuanku. Setelah aku setujui dan berhasil dijalankan, simpan SQL-nya ke supabase/migrations/fitur_a_work_order.sql dan perbarui src/types/database.ts.

PROMPT A2 *salin teks di bawah ini*

Lengkapi kerangka /work-orders, /work-orders/baru, dan /work-orders/[id] sesuai PRD 6.6, 6.7, dan 6.8 bagian A sampai C. Tombol status, termasuk Serahkan Mobil, hanya tampil untuk perpindahan yang sah bagi peran yang login dan memanggil ubah_status_wo lewat supabase.rpc(). Foto diperkecil sebelum diunggah dan memakai thumbnail (PRD 11.3 dan 11.4). Jangan mengubah folder tagihan dan invoice milik fitur B.

Uji Fitur A

- [ ] **Rina** membuat WO dari plat lama: data pelanggan terisi, nomor WO lanjut dari nomor terakhir.
- [ ] **Pak Joko** menunjuk Andi lalu Mulai Kerjakan: status menjadi dikerjakan.
- [ ] **Pak Joko** membuka WO milik Tim B: tombol status tidak tampil.
- [ ] Alamat foto STNK dengan /sign/ diganti /public/, dibuka tanpa login: ditolak.

PROMPT UJI A *salin teks di bawah ini*

Lewat execute_sql, tiru login Pak Joko lalu panggil ubah_status_wo untuk satu WO Tim B, dan untuk lompatan menunggu ke siap_diambil di WO Tim A. Jangan ubah data lain. Laporkan pesan error kedua uji.

Fitur B: Tagihan dan *Invoice*

B

Catat spare part dan jasa di WO, sesuaikan harga beralasan, terbitkan *invoice*, dan catat pembayaran.

Fitur B

Halaman yang dilengkapi	*Database function* yang dibuat

`/work-orders/[id]/tagihan`
`/work-orders/[id]/invoice`
	`tambah_item_tagihan, hapus_item_tagihan`
	`sesuaikan_harga_item, atur_diskon`
	`terbitkan_invoice`
	`catat_pembayaran`

Yang dilatih di fitur ini

Transaction: stok terakhir hanya bisa dipakai satu kali dan nomor *invoice* tidak pernah ganda, walaupun dua orang menekan tombol bersamaan.

Prompt Fitur B

PROMPT B1 *salin teks di bawah ini*

Fitur B. Lewat Supabase MCP, buat fungsi tambah_item_tagihan, hapus_item_tagihan, sesuaikan_harga_item, atur_diskon, terbitkan_invoice, dan catat_pembayaran sesuai PRD 9.2. Fungsi yang mengubah stok dan nomor invoice wajib mengunci baris dengan for update. Penyesuaian harga wajib beralasan. Tampilkan SQL dulu dan tunggu persetujuanku. Setelah aku setujui dan berhasil dijalankan, simpan SQL-nya ke supabase/migrations/fitur_b_tagihan.sql dan perbarui src/types/database.ts.

PROMPT B2 *salin teks di bawah ini*

Lengkapi kerangka /work-orders/[id]/tagihan dan /work-orders/[id]/invoice sesuai PRD 6.8 bagian D dan 6.9: Kepala Mekanik menambah dan menghapus item saat dikerjakan; Admin menyesuaikan harga dengan alasan wajib, dengan peringatan bila di bawah modal, dan mengatur diskon; Terbitkan Invoice; Catat Pembayaran sebagian atau lunas; invoice rapi saat dicetak. Semua aksi lewat supabase.rpc(). Jangan mengubah halaman milik fitur A.

Uji Fitur B

- [ ] **Pak Joko** menambah Lampu H4 (stok 1) dari dua tab bersamaan: satu berhasil, satu "Stok Lampu H4 hanya 0".
- [ ] **Rina** menyesuaikan harga tanpa alasan: ditolak.
- [ ] **Rina** menerbitkan *invoice* untuk WO siap diambil: nomor INV berikutnya, tagihan terkunci.
- [ ] Bayar sebagian, lalu lunas: status bayar berubah.
- [ ] *Invoice* dicetak rapi dengan Ctrl+P.

Fitur C: Inventory dan Pengguna

C

Kelola spare part, jasa, dan stok, lalu halaman Owner untuk mengatur akun dan setelan bengkel.

Fitur C

Halaman yang dilengkapi	*Database function* yang dibuat

`/inventory`
`/pengguna`
`/setelan`
	`catat_stok_masuk`
	`sesuaikan_stok`
	bucket publik untuk logo dan galeri

Yang dilatih di fitur ini

Harga modal hanya terkirim ke *browser* Owner. Stok hanya berubah lewat *database function* dan setiap perubahan tercatat di riwayat stok.

Prompt Fitur C

PROMPT C1 *salin teks di bawah ini*

Fitur C. Lewat Supabase MCP, buat fungsi catat_stok_masuk dan sesuaikan_stok sesuai PRD 9.2, serta bucket publik beserta policy-nya (PRD Bab 11). Fungsi mengunci baris item dengan for update. Tampilkan SQL dulu dan tunggu persetujuanku. Setelah aku setujui dan berhasil dijalankan, simpan SQL-nya ke supabase/migrations/fitur_c_inventory.sql dan perbarui src/types/database.ts.

PROMPT C2 *salin teks di bawah ini*

Lengkapi kerangka /inventory, /pengguna, dan /setelan sesuai PRD 6.4, 6.10, dan 6.11:
1. Inventory dengan tab Spare Part, Jasa, dan Riwayat Stok. Kolom harga modal dari inventory_costs hanya diambil dan ditampilkan untuk Owner. Tombol Stok Masuk untuk Admin dan Owner, Penyesuaian Stok untuk Owner.
2. Pengguna: Owner mengubah peran dan status aktif akun.
3. Setelan: Owner mengubah data bengkel, logo, dan galeri di bucket publik.
Daftar berhalaman 20 baris dan hanya mengambil kolom yang ditampilkan.

Uji Fitur C

- [ ] **Pak Darto** membuka Inventory: kolom harga modal tampil.
- [ ] **Rina** membuka Inventory: kolom modal tidak ada, dan di DevTools › Network tidak ada harga_modal.
- [ ] **Rina** mencatat stok masuk: stok bertambah dan tercatat di Riwayat Stok.
- [ ] **Pak Darto** menonaktifkan Pak Slamet: Pak Slamet tidak bisa masuk. Aktifkan lagi.
- [ ] **Pak Darto** mengunggah logo bengkel di Setelan.

Fitur D: *Dashboard* dan *Audit Log*

D

Tampilkan *dashboard* berbeda untuk tiga peran, catat setiap perubahan penting, dan buat laporan laba untuk Owner.

Fitur D

Halaman yang dilengkapi	*Database function* yang dibuat

`/dashboard`
`/audit-log`
`/laporan-laba`
	`isi supabase/dashboard.sql`
	`trigger` `catat_audit`

Yang dilatih di fitur ini

Audit log terisi otomatis lewat *trigger*. Angka *dashboard* dihitung di database supaya *egress* tetap kecil.

Prompt Fitur D: Database dan *Audit Log*

PROMPT D1 *salin teks di bawah ini*

Fitur D. Lewat Supabase MCP:
1. Jalankan isi supabase/dashboard.sql tanpa mengubahnya.
2. Buat trigger catat_audit di tabel PRD 9.4. Pastikan audit_logs tidak bisa diubah atau dihapus siapa pun. Tampilkan SQL trigger dulu dan tunggu persetujuanku. Setelah aku setujui dan berhasil dijalankan, simpan SQL-nya ke supabase/migrations/fitur_d_audit.sql dan perbarui src/types/database.ts.

PROMPT D2 *salin teks di bawah ini*

Lengkapi kerangka /audit-log dan /laporan-laba sesuai PRD 6.12 dan 6.13. Audit Log: 20 baris per halaman, penyaring tabel dan pelaku, serta perbandingan nilai sebelum dan sesudah. Laporan Laba memakai fungsi laporan_laba lewat supabase.rpc(). Keduanya hanya untuk Owner.

Prompt Fitur D: Tampilan *Dashboard*

PROMPT D3 *salin teks di bawah ini*

Lengkapi kerangka /dashboard sesuai PRD Bab 12 untuk tiga peran. Semua angka dan grafik diambil dari fungsi dashboard_* lewat supabase.rpc(), tidak dihitung dari seluruh baris.
1. Pilihan periode (Hari ini, 7 hari, 30 hari, Bulan ini) yang mengubah semua kartu dan grafik.
2. Kartu KPI dengan ikon, angka besar, dan perubahan dibanding periode sebelumnya (naik hijau, turun merah).
3. Grafik Recharts lewat komponen chart shadcn/ui: area untuk tren harian, batang untuk beban mekanik dan item terlaris, donat untuk WO per status dengan warna status PRD 7.1.
4. Setiap grafik punya judul, keterangan singkat, tooltip, dan tampilan kosong. Klik bagian grafik membuka daftar sumbernya.
5. Skeleton saat memuat, grid rapi di laptop dan tablet, warna dari globals.css.

Uji Fitur D

- [ ] Masuk dengan tiga peran: isi *dashboard* berbeda, Rina dan Pak Joko tidak melihat laba.
- [ ] Ganti periode: semua kartu dan grafik ikut berubah.
- [ ] Klik irisan status di grafik donat: membuka daftar WO dengan status itu.
- [ ] Ubah harga jual satu item, lalu buka *Audit Log* sebagai Pak Darto: tercatat dengan nilai sebelum dan sesudah.
- [ ] Laporan Laba hanya bisa dibuka Pak Darto.

ISOMA

Kembali pukul 13.00

Sebelum istirahat: simpan perubahanmu (*commit*).

Langkah Membuat *Pull Request*

14.15 · 30 menit

PROMPT PUSH *salin teks di bawah ini*

Tarik main terbaru ke branch-ku. Bila ada merge conflict, berhenti dan tunjukkan dulu. Bila aman, commit semua perubahan dengan pesan yang menjelaskan isinya, lalu push branch-ku ke GitHub. Tuliskan juga ringkasan perubahan dan cara mengujinya untuk deskripsi pull request.

1. Buka repo kelompok di GitHub, klik **Compare & pull request**.
2. Tempel ringkasan dari agen ke kolom deskripsi.
3. Di **Reviewers**, pilih satu teman kelompok.
4. Klik **Create pull request**, lalu sampaikan ke *reviewer*.

Langkah *Review* dan *Merge*

Reviewer

1. Buka PR, baca tab **Files changed**.
2. Kirim *prompt* coba di bawah, lalu uji fiturnya.
3. Tulis komentar, lalu **Review changes › Approve**.

Ketua

1. Pastikan PR sudah disetujui satu teman.
2. Klik **Merge pull request › Confirm merge**.
3. Sampaikan ke kelompok: "PR fitur B sudah masuk main."

PROMPT COBA *salin teks di bawah ini*

Simpan dulu pekerjaanku dengan commit. Lalu ambil branch fitur/tagihan dari GitHub dan pindah ke branch itu supaya aku bisa mencoba fitur temanku. Setelah selesai, kembalikan aku ke branch-ku sendiri.

TARIK MAIN *salin teks di bawah ini*

Tarik main terbaru dari GitHub ke branch-ku. Bila ada merge conflict, berhenti dan tunjukkan dulu.

Prompt Menyelesaikan *Merge Conflict*

PROMPT KONFLIK *salin teks di bawah ini*

Ada merge conflict. Untuk setiap file yang bentrok:
1. Tunjukkan bagian yang bentrok dan jelaskan isi kedua versinya dengan bahasa sederhana.
2. Usulkan penyelesaian yang mempertahankan pekerjaan kedua fitur.
3. Tunggu persetujuanku sebelum mengubah file.
Khusus src/types/database.ts, jangan digabung manual. Buat ulang lewat generate_typescript_types.
Setelah selesai, jalankan npm run build untuk memastikan tidak ada error.

Bila konflik ada di file milik fitur lain, ajak pemilik fiturnya memutuskan bersama.

Uji Alur Lengkap Bersama

14.45 · 30 menit

Setelah empat PR masuk main, setiap anggota masuk dengan **satu akun** di laptopnya, lalu jalankan satu kunjungan mobil:

1	Rina	Buat WO baru dari plat B 1234 KJ	☐
2	Pak Joko	Tunjuk mekanik, mulai, tambah aki, selesai, QC lulus	☐
3	Rina	Terbitkan *invoice*, catat pembayaran lunas	☐
4	Rina	Unggah foto kondisi akhir, Serahkan Mobil	☐
5	Pak Darto	Lihat pendapatan di *dashboard* dan jejaknya di *Audit Log*	☐

Apa Itu *Pair Testing*?



SOFT SKILL
Collaboration

DEFINISI

Pair testing adalah cara menguji aplikasi berdua: satu orang mencoba, satu orang mengamati dan mencatat.

Penguji	Pembuat
• Memegang laptop dan mencoba skenario • Mengucapkan apa yang sedang dilakukan • Menulis laporan temuan	• Menyiapkan aplikasi dan akun uji • Mengamati tanpa mengarahkan • Mencatat dan bertanya langkahnya

Aturannya: penguji memegang laptop, pembuat tidak membela diri saat celah ditemukan.

Bagaimana Menulis Laporan Temuan?



SOFT SKILL
Collaboration

Laporan yang baik membuat pembuat bisa mengulang masalahnya sendiri. Contohnya dua laporan untuk temuan yang sama:

✕ Sulit diulang

"Aplikasimu error di WO, coba cek deh."

✓ Bisa diulang

Akun	Pak Joko (Tim A)
Langkah	Buka WO-2610-0011 milik Tim B, klik Mulai Kerjakan
Seharusnya	Tombol tidak tampil atau ditolak
Terjadi	Status berubah menjadi dikerjakan
Bukti	Tangkapan layar halaman WO

Bagaimana Menanggapi Temuan dari Teman?

SOFT SKILL
Collaboration

Temuan dari teman adalah celah yang tidak jadi ditemukan klien. Contohnya kalimat saat menerima temuan:

Ucapkan	Hindari
"Terima kasih, boleh tunjukkan langkahnya?"	*"Di laptopku jalan kok."*
"Aku ulangi dulu ya, biar yakin."	*"Itu bukan bug, kamu salah pakai."*
"Nanti tolong uji lagi setelah kuperbaiki."	*"Ya sudah, nanti saja."*

Setelah itu: catat temuan, perbaiki, lalu minta teman menguji ulang.

Latihan: Uji Aplikasi Kelompok Lain

15.15 · 25 menit

Dua orang dari kelompokmu menguji aplikasi kelompok pasangan selama 15 menit. Tulis temuan dengan format laporan di lembar temuan, lalu serahkan ke kelompok pemilik.

	Skenario	Akun	Hasil yang benar
1	Buka WO milik Tim B dan coba ubah statusnya	**Pak Joko**	**Tidak bisa**
2	Ketik /audit-log dan /laporan-laba di alamat	**Rina**	**Dialihkan**
3	Cari harga modal di Inventory dan DevTools › Network	**Rina**	**Tidak ada**
4	Tambah stok terakhir dari dua tab bersamaan	**Pak Joko**	**Satu ditolak**
5	Serahkan mobil sebelum *invoice* lunas	**Rina**	**Ditolak**

Langkah Menayangkan Aplikasi

15.40 · Ketua

PROMPT TAYANG *salin teks di bawah ini*

Pindah ke branch main dan tarik versi terbaru. Tayangkan aplikasi ke Netlify lewat Netlify CLI dengan nama situs cemerlang-jaya-k1. Isi NEXT_PUBLIC_SUPABASE_URL dan NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY dari .env.local ke environment variable Netlify tanpa menampilkan nilainya. Setelah selesai, beri aku URL-nya.

1. Buka URL Netlify, masuk sebagai Pak Darto.
2. Di Supabase, buka **Authentication › URL Configuration**, ganti Site URL dengan URL Netlify.
3. Serahkan URL ke trainer.
4. Seluruh anggota mencoba masuk dari laptop masing-masing.

Eksplorasi Mandiri

Perbaiki temuan	Pemilik fitur memperbaiki temuan uji silang lewat *branch* dan *pull request*.	Kelompok
Halaman depan	Logo, layanan, dan galeri dari *bucket* publik tanpa login (PRD 6.2).	Kelompok
Tayang ulang	Ketua menayangkan ulang main setelah semua perbaikan masuk.	Ketua
Login Google	Ikuti Modul Bacaan 6 Sesi 8.	Eksplorasi

Kirim URL Netlify dan link repo kelompok di LMS

Terima Kasih!

Aplikasi bengkel kelompokmu sudah tayang, dibangun bersama lewat pull request.



Cemerlang Jaya Motor

TAYANG